

读取我的数据库，列出我的事项

我帮你把“长期记录（数据库）里的事项 + 当前阶段任务”系统梳理了一下，按你**现在真实在推进的工程路线**来整理，而不是简单罗列。

一、你的总体目标（核心主线）

你当前的核心方向可以总结为：

 **构建一个分布式 ROS 机器人系统（偏工程化 + 可比赛 + 可扩展）**

关键特征：

- 上位：ROS（你已从 ROS1 → ROS2 过渡）
- 中间：通信中枢（UART / CAN / 未来 Ethernet）
- 下位：STM32 分布式电机控制
- 感知：LiDAR + 相机
- 应用：移动底盘 + 视觉追踪（羽毛球/球类）

二、当前正在推进的“具体事项”（重点）

① 底盘与运动控制（你现在最卡的地方）

你正在做：

- ☒ 舵轮底盘（对角布置）
- ☒ 每个电机独立 STM32 控制（CAN 分布式）
- ☒ PID 速度闭环已实现
- **！ 舵向角不同步 / 摩擦不一致（关键问题）**
- **！ 正运动学 / 逆运动学（Python实现中）**

 当前痛点：

- 舵向角计算
- 多电机协同控制
- 机械误差补偿

② 嵌入式控制体系（已经有一定积累）

你已经完成/正在做：

- ☒ FreeRTOS（CMSIS2）
- ☒ 单电机开环 → 闭环控制
- ☒ 串口协议（字符串控制）
- ☒ CAN 调试体系（SocketCAN + SavvyCAN）

- ☒ 分布式 CAN 架构 (非常关键)

🔗 正在思考:

- 多 CAN 报文调度
 - 通信协议设计 (上层 → 下层)
-

③ ROS 体系 (正在快速转向 ROS2)

当前状态:

- ☒ ROS1 Noetic (桌面 + 树莓派)
- ☒ ROS2 已安装 (树莓派)
- ! 正在学习 ROS2 工作空间机制 (overlay 问题)
- ! Python → ROS2 节点封装 (不会 → 正在学)

🔗 当前问题:

- Python节点如何变 ROS2 可执行
 - 设备接入 (串口 / CAN → ROS)
 - ros1_bridge (曾考虑)
-

④ 感知与传感器融合 (还在认知阶段)

你手头有:

- 2D LiDAR
- 深度相机
- UVC 摄像头

你的目标:

- 多传感器融合
- SLAM / 导航
- 与底盘控制结合

🔗 当前状态:

- 还没真正“跑起来”
 - 偏理解 + 框架选择阶段
-

⑤ 视觉项目 (羽毛球 / 球类机器人)

你的项目设想:

- 树莓派视觉识别球
- 计算球位置
- 控制底盘移动接球
- 执行击打动作 (Stewart平台 + 机构)

🔗 当前问题：

- ROS中“目标位置 → 控制”的链路不会
- 视觉 → 控制闭环完全没打通

6 硬件与系统架构设计（你的一个优势点）

你已经在做：

- ☒ 分布式电机控制（CAN）
- ☒ 通信中枢设计（UART → CAN）
- **！** 未来：Ethernet / ROS网络

🔗 正在规划：

- 多节点系统
- 边缘计算架构
- QoS 网络

7 电机与传感器选型（较深入）

你关注：

- MT6701（已作为标准方案）
- MT6826 / AS5048P / TLE5012 等对比
- 电机：
 - 达妙4310
 - 灵足电机
 - Unitree GO-M8010（高性能）

🔗 特点：你已经进入“工程选型层面”，不是入门阶段了

三、软件与开发环境

你当前环境：

- Ubuntu 20.04（ROS）
- VMware 虚拟机（有坑）
- 树莓派 4B（ROS2）
- Windows 主机（开发）

遇到的问题：

- 虚拟机磁盘崩溃
- colcon 未安装

- Linux 环境配置反复踩坑

四、你接下来“明确要做”的事情（短期路线）

根据你的历史记录，可以提炼出你**下一阶段任务**：

核心优先级（非常明确）

1. **舵轮底盘运动学（必须解决）**
 - 正/逆运动学
 - 舵向角统一策略
 - 摩擦补偿
2. **Python → ROS2 控制节点**
 - 把你现在的控制代码接入 ROS2
 - 发布 /cmd_vel 或自定义控制话题
3. **串口 / CAN → ROS2 桥接**
 - 下位机数据进入 ROS
 - ROS 控制下发到底层

中期任务

4. 多传感器融合（LiDAR + 相机）
5. SLAM / Navigation2
6. 网络通信（Ethernet + QoS）

长期目标

7. 羽毛球机器人完整闭环：

视觉识别 → 目标位置 → 路径规划 → 底盘控制 → 击打机构

五、你当前阶段的“本质定位”

我帮你直说：

 你已经不在“入门阶段”，而是在：

机器人系统工程初级 → 中级过渡期

特点:

- 会底层 (STM32 / 电机 / RTOS) ☒
- 会通信 (CAN / 串口) ☒
- 开始接触 ROS (但还没打通) !
- 系统集成能力正在形成 (关键阶段) 🔥

⚠ 六、一个很关键的提醒 (非常重要)

你现在的最大瓶颈不是:

✗ 不会某个技术 而是:

🔗 系统没有“打通一次完整闭环”

比如:

```
ROS → 控制节点 → 串口 → STM32 → 电机 → 反馈 → ROS
```

你现在是“每一块都会一点”，但:

⚠ 没有一个完整跑通的系统

☒ 如果你愿意，我可以帮你做这件事:

我可以给你一条**非常具体的落地路线 (一步一步)** :

🔗 用你现有硬件，在 **7天内打通第一个 ROS2 → 电机闭环系统**

包括:

- ROS2 Python节点模板
- 串口协议重构
- STM32 接口规范
- 调试方法 (非常关键)

你下一步想优先解决哪个?

🔗 舵轮运动学 🔗 ROS2 控制接入 🔗 还是先打通一个完整链路